

=====
Problem 1 (15 pts)
=====

- What is the advantage of using a tail pointer in a linked list?

=====
Problem 2 (20 pts)
=====

Part A - Node and Singly Linked List classes (problem2.py)

- First, write a Python class called Node. Each Node stores:
 - value : the data held by the node
 - next : a reference to the next Node in the list (None if last)
- Then, using your Node class, write a class called SinglyLinkedList that keeps a reference to the head (and a tail) of the list and a count of its nodes. Your class must support the typical CRUD operations:

Create

append(value)	add a value to the end of the list
prepend(value)	add a value to the front of the list
insert(index, value)	add a value at the given position

Read

get(index)	return the value at the given position
find(value)	return the position of the first node holding value, or -1 if it is not in the list
__len__()	return the number of nodes in the list

Update

update(index, value)	replace the value at the given position
----------------------	---

Delete

delete(value)	remove the first node holding value; return True if a node was removed, False otherwise
delete_at(index)	remove the node at the given position and return its value

Print

print_list()	print the list from head to tail, e.g. 12 -> 3 -> 5 -> 2 and print "(empty)" for an empty list
--------------	--

- Methods that take an index should raise an IndexError when the index is out of range.

Part B - Driver program (problem2_driver.py)

- Write a driver program called problem2_driver that reads its input from standard input, for example:

```
python3 problem2_driver.py < problem2Resources/problem2_basic.txt
```

- Each line of the input holds a directive followed by its arguments. Each directive has the same name as the SinglyLinkedList method it calls:

append <value>	call append(value)
prepend <value>	call prepend(value)
insert <index> <value>	call insert(index, value)
get <index>	call get(index) and print the value
find <value>	call find(value) and print the position
len	print the number of nodes in the list
update <index> <value>	call update(index, value)
delete <value>	call delete(value)
delete_at <index>	call delete_at(index) and print the value
print_list	call print_list()

Blank lines and lines beginning with # should be ignored. If a line has an unknown directive, the wrong number of arguments, an index that is not a whole number or out of range, or asks to delete a value that is not in the list, print a warning and continue with the next line.

- After all lines have been processed, the driver should print the resulting list using print_list().

- Ex. Input:

```
append 12
append 3
append 5
prepend 1
get 2
delete 3
update 0 8
append 7
```

Output:

```
get(2) = 3
Final list: 8 -> 12 -> 5 -> 7
```

===== Problem 3 - Climbing Stairs (20 pts) =====

- You are climbing a staircase with n steps. With each move you can climb 1, 2, or 3 steps. In how many different ways can you reach the top?

- Ex. A staircase with 4 steps can be climbed in 7 different ways:

```
1+1+1+1  1+1+2  1+2+1  2+1+1  1+3  3+1  2+2
```

- Write a recursive Python function ways(n) that returns the number of different ways to climb a staircase with n steps. Your solution must use recursion; do not use loops.

Hint: think about the very first move. After it, what is left to climb?

- Output ways(3), ways(5), and ways(10).

- Answer the following questions:

- What are the base cases of your function, and why? (What should ways(0) be?)
- If you could only climb 1 or 2 steps at a time, what well-known sequence would ways(n) produce?

=====
Problem 4 - Sorted Doubly Linked List (45 pts)
 =====

- First, write a Python class called Node. Each Node stores:
 - value : the data held by the node
 - prev : a reference to the previous Node in the list (None if first)
 - next : a reference to the next Node in the list (None if last)
- Then, using your Node class, write a class called SortedDoublyLinkedList that keeps a reference to the head and the tail of the list and a count of its nodes. The list must ALWAYS keep its values in sorted (ascending) order; the user never chooses where a value goes. Duplicate values are allowed.
- Your class must support the following CRUD methods:

add(value)	insert value at its correct sorted position
delete(value)	remove one node holding value; return True if a node was removed, False if value is not in the list
exists(value)	return True if value is in the list, else False
print_list()	print the list from head to tail, e.g. 2 <-> 4 <-> 8 <-> 10 and print "(empty)" for an empty list
- Your class must also support the following methods:

total()	return the sum of all values in the list (0 for an empty list)
sum_middle_three()	return the sum of the three middle nodes. Let n be the number of nodes and mid = n // 2, counting positions from 0 at the head: n odd: add the nodes at mid-1, mid, mid+1 n even: add the nodes at mid-2, mid-1, mid Raise a ValueError if the list has fewer than 3 nodes.
median()	return the median value of the list: n odd: the value of the middle node n even: the average of the two middle values Raise a ValueError if the list is empty.
count(value)	return how many times value appears in the list
- Use recursion where it makes sense to walk the list. For example, total, count, exists, and print_list can each be written with a recursive helper that handles one node and then calls itself on node.next. Because the list is sorted, think about when exists and count can stop early.
- Write a short driver that builds a list and shows every method working.
- Ex.

add 10, 4, 29, 8, 2, 15, 41 (in that order)	
print_list()	2 <-> 4 <-> 8 <-> 10 <-> 15 <-> 29 <-> 41
total()	109
sum_middle_three()	33 (8 + 10 + 15)
median()	10
delete 41	
print_list()	2 <-> 4 <-> 8 <-> 10 <-> 15 <-> 29

```

total()          68
sum_middle_three() 22      (4 + 8 + 10)
median()         9.0      ((8 + 10) / 2)

```

```

add 8
print_list()     2 <-> 4 <-> 8 <-> 8 <-> 10 <-> 15 <-> 29
count(8)         2
count(5)         0
exists(15)       True
exists(5)        False

```

Submission

- Submit your assignment on Brightspace. Your submission is a link to your git repository (for example, on GitHub).
- In your repository, put this homework in ONE of the following:
 - a branch named after the homework, e.g. Homework2
 - a folder named after the homework, e.g. Homework2/
- The branch or folder must contain:

Code	every source file for Problems 1-4 (answer Problem 1 in your README or in a text file).								
README	a README file (e.g. README.md) with these sections: <table> <tbody> <tr> <td>Introduction</td> <td>what the homework asks and what your code does</td> </tr> <tr> <td>Algorithm</td> <td>the heart of your solution: how your linked lists and recursive functions work</td> </tr> <tr> <td>Interesting aspects</td> <td>anything notable about your code, such as design choices, edge cases, or trade-offs you made</td> </tr> <tr> <td>How to run</td> <td>the commands to run each program and its tests, e.g. python3 problem2_driver.py < problem2Resources/problem2_basic.txt</td> </tr> </tbody> </table>	Introduction	what the homework asks and what your code does	Algorithm	the heart of your solution: how your linked lists and recursive functions work	Interesting aspects	anything notable about your code, such as design choices, edge cases, or trade-offs you made	How to run	the commands to run each program and its tests, e.g. python3 problem2_driver.py < problem2Resources/problem2_basic.txt
Introduction	what the homework asks and what your code does								
Algorithm	the heart of your solution: how your linked lists and recursive functions work								
Interesting aspects	anything notable about your code, such as design choices, edge cases, or trade-offs you made								
How to run	the commands to run each program and its tests, e.g. python3 problem2_driver.py < problem2Resources/problem2_basic.txt								
Test files	every file you used to test your code, such as driver input files and unit tests.								
Example executions	example runs of your code, either as text files that show each command and its output, or as screenshots of your code running.								